

Nimble-RX Onboarding

Tyler Dinh

September 20, 2026

1 Overview

Nimble-RX is a pharmacy robotics project focused on building an autonomous robot capable of interacting with a pharmacist, perceiving its environment, and manipulating prescriptions (e.g., pill bottles). This document provides resources for members to get up to speed with different aspects of the project.

2 Software: Interaction

The Interaction (Interface) subsystem enables the robot to communicate back and forth naturally with pharmacists via STT and TTS and provides a frontend for monitoring and control.

2.1 Speech-to-Text

The robot must receive natural language commands from the pharmacist (e.g., “Pick up Prescription A, Prescription B”). A low-latency, seamless STT pipeline is critical.

Key considerations:

- Streaming inference for real-time responsiveness
- Robustness to background noise

Resources:

- Parakeet STT: <https://github.com/openai/whisper>
- “An Edge-Enabled Low-Latency Cross-Lingual Speech-to-Text Framework for Humanoid Robots” (2026): <https://journals.sagepub.com/doi/10.1177/2167647X261438099>

2.2 Text-to-Speech

The robot must confirm actions verbally (e.g., “You requested me to pick up Prescription A and Prescription B, would you like me to pick up anything else?”). Again, low latency is essential, so we will use models optimized for this metric (such as those that are small with high performance).

Resources:

- Kokoro TTS: <https://huggingface.co/hexgrad/Kokoro-82M>

2.3 LLM Fine-Tuning for Conversational Interaction

A fine-tuned LLM used as the robot’s brain can make robot-to-pharmacist dialogue more natural and context-aware.

Resources:

- “When Robots Get Chatty: Grounding Multimodal Human-Robot Conversation” (2024): <https://arxiv.org/html/2407.00518v1>
- “Understanding Large-Language Model (LLM)-powered Human-Robot Interaction” (2024): <https://peopleandrobots.wisc.edu/wp-content/uploads/sites/1469/2025/02/3610977.3634966.pdf>
- Hugging Face LLM Fine-Tuning Guide: https://huggingface.co/docs/autotrain/llm_finetuning

2.4 Frontend Interface

A user-facing web application (TypeScript/React) for the pharmacist to monitor robot status, view task queues, and override actions. The backend communicates with the robot. Also responsible for interface-related safety tasks such as emergency stop buttons and confirmation dialogs.

Approach:

- A TypeScript-based frontend provides a responsive UI accessible from a pharmacist’s desktop or mobile device.
- Real-time robot status updates via WebSockets.
- Safety-critical actions (emergency stop) require explicit user confirmation.
- The frontend can also serve as a data collection interface for labeling and verifying teleoperation episodes.

Resources:

- “A Framework for Low-Latency, LLM-Driven Multimodal Interaction on the Pepper Robot” (2026): <https://arxiv.org/html/2603.21013v1>

3 Software: Perception

A robot should verify the environment and its contained objects so that (1) it does not hallucinate and (2) it can correct its hallucinations when they occur.

General Resources:

- PaddleOCR – lightweight, multilingual OCR: <https://github.com/PaddlePaddle/PaddleOCR>
- “An Accurate Deep Learning-Based System for Automatic Pill Identification” (2023): <https://pmc.ncbi.nlm.nih.gov/articles/PMC9883737/>
- Cognex OCR for prescription scanning: <https://www.cognex.com/en/applications/optical-character-ocr-for-prescription-scanning>

3.1 DataMatrix Barcode Decoding

DataMatrix barcodes on prescriptions encode metadata. The robot must decode these to answer “What did we just pick up?”

Resources:

- OpenCV - main computer vision library in Python for images / videos: <https://opencv.org>
- pylibdmtx – Python library for reading/writing DataMatrix codes: <https://pypi.org/project/pylibdmtx/>
- GS1 DataMatrix White Paper (NCPDP): <https://www.ncdp.org/NCPDP/media/pdf/WhitePaper/GS1-DataMatrix-White-Paper-v1.pdf>
- “How to Make a Barcode Reader in Python” (GeeksforGeeks): <https://www.geeksforgeeks.org/python/how-to-make-a-barcode-reader-in-python/>

3.2 Safety Considerations

Perception-related safety includes verifying that the robot has identified the correct pill bottle before grasping, and cross-referencing OCR/barcode outputs against expected prescriptions.

4 Software: Interaction

Given a natural language command and image(s) of the environment, the robot outputs an *action chunk*. This output is a sequence of action vectors describing deltas for end-effector actions, which drives low-level control.

4.1 Vision-Language-Action (VLA) Models

VLAs unify vision, language, and action modalities to produce low level robot control commands. This is the core of our work.

Resources:

- “Vision-Language-Action Models for Robotics: A Review Towards Real-World Applications” (2025): <https://ieeexplore.ieee.org/document/11164279>

4.2 NORA 1.5

NORA is a 3B-parameter VLA model built on Qwen-2.5-VL-3B. NORA 1.5 integrates a flow-matching action expert with an autoregressive backbone and uses DPO with world-model and action-based preference rewards on top of the base NORA model. It achieves SOTA performance on several benchmarks while its inference is possible on consumer GPUs (maybe even your laptop?).

Resources:

- “NORA-1.5: A Vision-Language-Action Model Trained using World Model- and Action-based Preference Rewards” (2025): <https://arxiv.org/abs/2511.14659>

4.3 High-Level planning on top of a low-level planning loop

There is some interesting work to be done in placing a high-level LLM/VLM planner above the VLA low-level planner. The high-level planner could decompose tasks (e.g., “fill prescription X”) into sub-tasks, while the VLA handles the actions needed for motor control.

Resources:

- “MEMORA: Embodied Action Memory from Egocentric Videos for Reasoning and Planning” (2026): <https://arxiv.org/pdf/2607.14252>

4.4 Teleoperation Data Collection

Training and fine-tuning VLAs require high-quality demonstration episodes of (prompt, images, action output) collected via human teleoperation.

Resources:

- LeRobot (Hugging Face) – framework for data collection and policy training: <https://github.com/huggingface/lerobot>

4.5 Safety Considerations

Interaction-related safety includes verifying that generated actions are within joint limits and workspace bounds, detecting collisions before execution, and implementing fallback behaviors when the VLA produces uncertain outputs.

5 Integration

The Integration subsystem connects software outputs to physical hardware, enabling the robot to move and sense its environment.

5.1 Linear Actuator

A long-standing project to enable the robot to move forwards and backwards between pharmacy shelves. This is essential since the robot is not a fully humanoid platform.

Resources:

- Adafruit A4988 breakout guide (wiring, VREF/current-limit trimming, power supply notes): <https://learn.adafruit.com/adafruit-a4988-stepper-motor-driver-breakout-board/circuitpython-and-python>
- StepperOnline: connecting a stepper motor to a Raspberry Pi (basic wiring reference): <https://help.stepperonline.com/en/article/how-to-connect-stepper-motor-with-raspberry-pi-ibu7oz>
- A4988 + Raspberry Pi walkthrough with current-limiting and STEP/DIR wiring: <https://whatibroke.com/2019/08/10/a4988-stepper-motor-raspberrypi/>
- RpiMotorLib – Python 3 library with an A4988-specific driver class (handles STEP/DIR/microstepping pins, acceleration ramping): <https://github.com/gavinlyonsrepo/RpiMotorLib>
- Controlling stepper motors with Python on a Pi (simpler from-scratch GPIO example, good for understanding what the library abstracts): <https://keithweaverca.medium.com/controlling-stepper>

5.2 Load Sensor (HX711)

A load sensor determines if the picked-up bottle has the correct weight (based on pill count), serving as a safety check.

Resources:

- SparkFun HX711 Load Cell Amplifier Hookup Guide: <https://learn.sparkfun.com/tutorials/load-cell-amplifier-hx711-breakout-hookup-guide>

- hx711-rpi-py – Python bindings for Raspberry Pi: <https://pypi.org/project/hx711-rpi-py/>

Notes for now:

- Connect the HX711 to a Raspberry Pi.
- Calibrate with a known weight to determine the scale factor.
- Design a PCB board integrating strain gauges, the HX711, and a connector for the Raspberry Pi.
- 3D-print a mounting bracket and use aluminum for the load cell platform.

6 Environment

A quiet (low-noise) environment is necessary for perception systems to function correctly. Additionally, the robot needs shelves and objects to interact with.

6.1 Shelf Design

Design or source a shelf for the robot to interact with pill bottles and prescriptions, mimicking a real pharmacy setting.

Resources:

- just look at what pharmacies do

6.2 Prescription Proxy Objects

Items in different colors, shapes, and sizes to act as prescriptions on the shelf. These are picked up by the robot during training and evaluation.

Resources:

- just different things you think of that could be picked up!

7 Prescription

The Prescription subsystem enables the robot to physically interact with prescriptions (more of a focus on pill bottles right now).

7.1 Pill Bottle Opener

A platform that presses down while spinning to open pill bottles, functioning similarly to an automatic can opener.

Approach:

- Design a compliant mechanism that applies downward force while rotating.
- Use a servo or stepper motor for rotational control.
- Implement torque sensing to detect when the cap has broken free.
- Consider 3D-printed adaptive grippers for different cap sizes.

Resources:

- RoboPharma: <https://www.robopharma.com/technology/kirby-lester-inventory-counter/>

7.2 Pill Bottle Closer

After the robot opens a bottle, it must reclose it. This requires aligning with the threading, which may involve:

- Force/torque feedback to detect thread engagement.
- A rotational + axial actuation system.
- Computer vision to verify cap orientation before closing.

7.3 Funnel Counting System

A funnel-based system that counts pills as they pass through, cutting off flow after a prescribed amount.

Resources:

- RoboPharma: <https://www.robopharma.com/technology/kirby-lester-inventory-counter/>